Adaptive Scheduling under Uncertain Processing Times: A Reproducible Empirical Study

Research engineering note | Independent empirical study

Abstract

Processing-time uncertainty creates a gap between optimized plans and realized execution. This independent research engineering study separates forecasting from replanning in a reproducible job-shop simulation. An exact-time execution engine exposes only public observations to fixed nominal or machine-level exponential moving average forecasts. A shared residual CP-SAT planner supports periodic, event and forecast-threshold triggers under an allocated search-effort cap. A final pilot comprises 64 policy runs on a tiny diagnostic instance and one public benchmark, across four duration regimes and two seeds. Results are mixed: forecast updating can improve or worsen realized makespan, while event-triggered planning frequently exhausts its budget and falls back to dispatching. Independent timeline reconstruction, forecast attribution and action replay support the reported measurements. The study establishes an inspectable experimental foundation; it does not establish a universally superior policy or propose a new scheduling algorithm.

1. Introduction

Forecast accuracy and scheduling effectiveness are different objectives. A forecast changes machine ordering and future decision opportunities; repeated optimization also consumes scarce computation. The research question is: how do processing-time updates and trigger mechanisms affect realized makespan when available information and configured optimization budgets are controlled? The project targets reproducible AI and operations research experimentation rather than algorithmic novelty.

2. Methodology

Jobs contain precedence-constrained operations on fixed machines, with capacity one and no preemption [1,2]. Private scenarios own future arrivals and realized durations. The simulator publishes completed events and current machine occupancy. Residual planning excludes completed work and freezes running machines and historical starts. Integer interval variables, precedence constraints and machine NoOverlap minimize forecast makespan. Proposed starts must still pass actual predecessor and idle-machine checks; a predicted finish never releases a physical resource.

Forecasts and commitments

Nominal forecasts remain fixed. For completed operation i on machine m, r = actual duration / nominal duration; the EMA update is a_m <- 0.25 r + 0.75 a_m, initialized at one [3]. Updates use only public completions, ordered deterministically. Predicted total duration is ceil(nominal x a_m), minimum one. A running operation has estimated residual max(1, estimated total - elapsed), not a known finish. Committed source forecasts are retained separately from updated start-time forecasts.

Policies

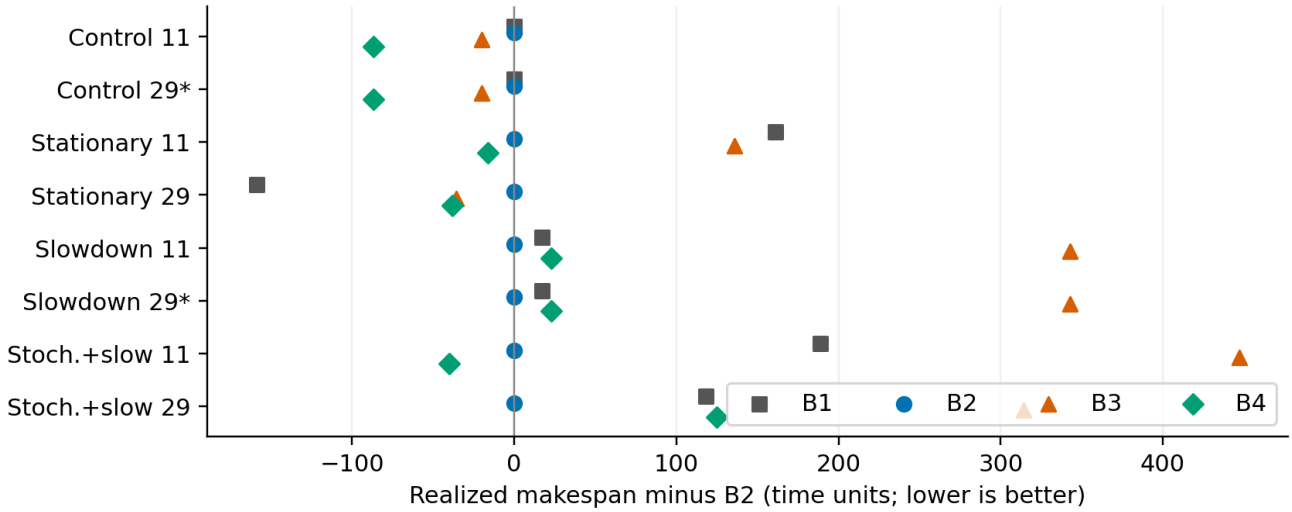
B0 is release-time FIFO and serves as an earlier diagnostic reference. The final comparison uses B1 (nominal, periodic), B2 (EMA, periodic), B3 (EMA, public completion/arrival), and B4 (EMA, forecast-change threshold). B4 uses strict relative change >0.25 with cooldown and explicit new-work integration. B2-B1 varies forecasting; B3-B2 and B4-B2 hold EMA fixed. Fallback and consumed computation remain parts of the resulting policy effects.

3. Experimental design

The final pilot uses a five-operation diagnostic and public abz5 (10 jobs, 10 machines, 100 operations), taken from the pinned TU/e benchmark checkout [1]. Four regimes are nominal control, operation-keyed stochastic multipliers in {1,2}, deterministic machine-0 slowdown, and their stochastic combination. Slowdown doubles duration for starts at/after time 3 (tiny) or 200 (abz5). Seeds are 11 and 29. All pilot jobs arrive at zero; separate tests exercise arrivals. One optimizer repetition yields 2 instances x 4 regimes x 2 seeds x 4 policies = 64 attempts. Deterministic seed duplicates are not independent realizations.

Computation and evaluation

OR-Tools 9.11.4210 uses one worker and seed zero. Per-call deterministic limit is 0.02 effort units, cumulative allocated cap 0.20 units/run, with a separate 10-second wall safety limit. Period and cooldown are 3/200 by instance. Equal caps do not imply equal actual work. Primary outcome is realized makespan. Secondary measures include solver cost, fallback, and separate start-time and authorizing-plan-time MAE/RMSE. Fallback operations have no invented plan-time prediction coverage.



abz5 only; one optimizer run per cell. *Same deterministic physical realization as seed 11.

Figure 1. All eight abz5 cells, relative to B2. Lower is better. Asterisks mark duplicated deterministic physical realizations; no confidence intervals are implied.

4. Results: forecasting is not uniformly beneficial

On stationary abz5, B2-B1 is -161 for seed 11 and +159 for seed 29. With stochastic slowdown the differences are -189 and -118. The tiny makespans tie for B1/B2. Thus favorable forecast-treatment outcomes coexist with a substantial negative outcome. Decision quality cannot be inferred from aggregate prediction error alone. Moreover, 221 of 812 planned B2 starts (27.2%) have a start-time prediction different from their authorizing plan forecast; the two error metrics describe different information sets.

Results: triggers interact with the budget

B3-B2 equals +343 on deterministic slowdown and +447/+314 on stochastic slowdown in abz5. B4-B2 is -40/+125 for stochastic slowdown: threshold triggering also has mixed outcomes. Across the final pilot, B3 performs 111 solver calls versus B2 92 and B4 55. Actual deterministic effort totals are 1.604, 0.656 and 0.453 units, respectively. These totals aggregate heterogeneous diagnostic runs and are descriptive, not equal-compute estimates.

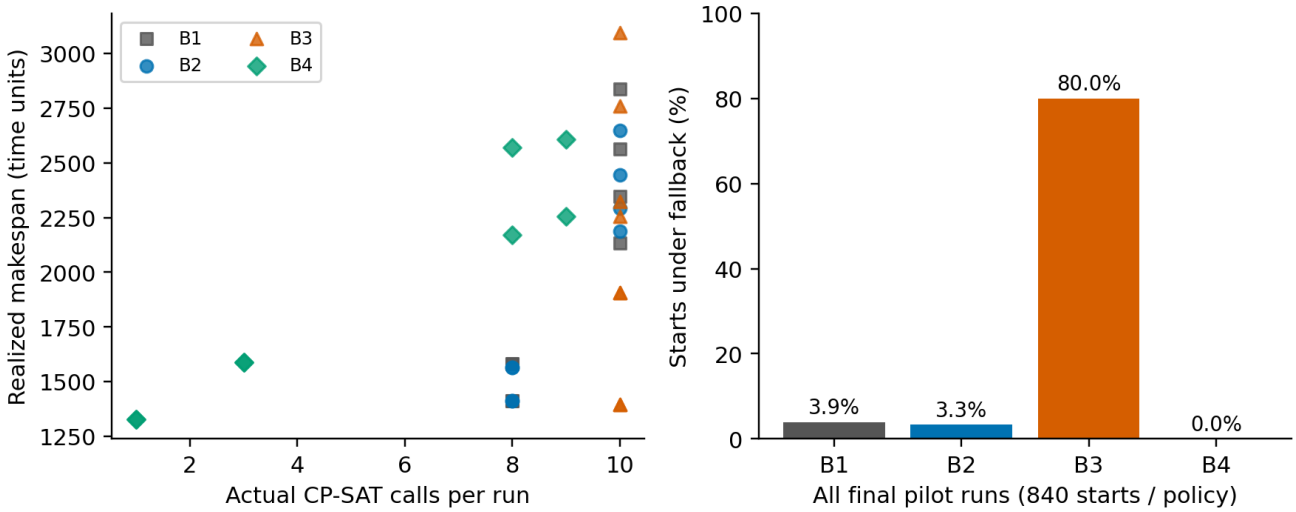


Figure 2. Left: abz5 makespan versus actual solve count. Right: operation-weighted fallback fraction across all 64 final runs, 840 starts per policy. More calls do not establish better execution.

Fallback interpretation

B3 starts 672/840 operations under fallback (80.0%), across eight fallback episodes and 3,846 callbacks. These counts have different denominators. Budget-exhaustion callbacks are not solver failures; FEASIBLE incumbents are not infeasible outcomes. Consequently, this is a comparison of budgeted hybrid policies, not the isolated effect of trigger frequency. Calendar-time slowdown also produces different realized durations for some operations across policies despite identical scenario manifests.

5. Validation, limitations and conclusion

All 64 final runs pass independent physical reconstruction and exact recorded-decision replay. The regression suite passes 130 tests; these support implementation checks, not scheduling superiority. Historical wall-limited reoptimization divergence remains documented; deterministic effort does not guarantee cross-platform bitwise results. Public coverage is limited to one benchmark instance and two seeds, with no significance claims. Forecast-error samples are policy-dependent, and fallback coverage differs. Python isolation is cooperative rather than a security sandbox. The platform offers inspectable simulation, optimization and experimental controls for subsequent research; broader effectiveness requires a separately preregistered study. No publication-level methodological novelty, institutional collaboration or HKU endorsement is claimed.

References

[1] AI for Decision Making @TU/e. Job Shop Scheduling Benchmark Environments and Instances. GitHub repository, pinned d088a26582f5cb79c6b4f2ad94f68ac7c50878cb (accessed 12 November 2026). [2] Google. OR-Tools: The Job Shop Problem. developers.google.com/optimization/scheduling/job_shop (accessed 12 November 2026). [3] NIST/SEMATECH e-Handbook of Statistical Methods, section 6.3.2.4, EWMA Control Charts. itl.nist.gov/div898/handbook/pmc/section3/pmc324.htm (accessed 12 November 2026).